Mid-Term Project Technical Report

Part 1: Common Section

1.1. Team Members

- Andrés Vidal Mira
- Serigne Mansour Diop

1.2. Title and Assignment

- **Title:** Implementation, Analysis, and Evaluation of Aesthetic Image Filter Effects.
- **Assignment:** The objective of this project is to design, implement, and demonstrate a set of image filters that produce visually distinctive and aesthetically meaningful effects. Unlike basic implementations, all filters must be developed **from first principles**, without relying on built-in high-level functions. Additionally, the project aims to explore **how aesthetic image transformations can be evaluated**, combining both mathematical metrics and perceptual interpretation. The final result includes an interactive application allowing real-time parameter experimentation.

1.3. Description of the Solution and Current Progress

Our solution is based on the implementation of four aesthetic filters, each representing a different category of image transformation:

- **Spatial domain filters (manual convolution):**
  - Emboss filter
  - Sharpen filter
- **Color transformation:**
  - Posterize filter
- **Gradient-based edge detection (manual implementation):**
  - Sobel Sketch filter

Key Improvement (Based on Feedback)

All filters are now implemented **manually**, without using built-in OpenCV functions such as cv2.Sobel or cv2.filter2D.

Instead, we:

- Define kernels explicitly
- Implement convolution using NumPy operations
- Compute gradients manually for edge detection

Current Progress

- Mathematical formulation of all filters is completed
- Initial implementations from scratch are functional

- Static testing environment (NumPy + Matplotlib) is completed

Final Objectives
The final system will include:

- A **real-time interactive application**
- Parameter control using sliders (intensity, thresholds, color levels)
- Visual comparison (original vs processed image)
- Optional visualization tools:
    - Histogram analysis
    - Intermediate steps (edges, gradients, etc.)

1.4. Description of the Data

We use standard test images to evaluate filter behavior under different visual conditions:

- **fit.jpg**
  Used to analyze geometric structures and strong edges. Particularly useful for evaluating emboss and edge-based filters.
- **lena.png**
  A standard reference image used to evaluate:
    - Detail preservation
    - Smooth gradients
    - Effects on human facial features
- **garden.jpg**
  Used to test filters on:
    - High texture density
    - Complex natural patterns

1.5. Division of Labor

Andrés Vidal Mira

- Implementation of spatial filters (Emboss, Sharpen)
- Development of convolution algorithms from scratch
- GUI development and interaction system

Serigne Mansour Diop :

- Implementation of Posterize and Sobel Sketch filters
- Manual gradient computation and thresholding
- Development of evaluation methodology
- Documentation and report structuring

Part 2: Individual Section – Andrés Vidal Mira

2.1. Summary of Studied Knowledge

The work focused on **spatial filtering and convolution**.

A digital image can be interpreted as a discrete function, and filters are applied using **convolution operations**, where a kernel (matrix) is applied over local neighborhoods.

- **Emboss Filter**
  - Based on directional differences in intensity
  - Creates artificial shadows and highlights
  - Enhances perception of depth
- **Sharpen Filter**
  - Amplifies high-frequency components
  - Enhances edges by emphasizing pixel differences
  - Improves perceived clarity and detail

2.2. Planned Technologies and Libraries

- Python
- NumPy (matrix operations and manual convolution)
- OpenCV (image loading and GUI only)
- Matplotlib (testing and visualization)

2.3. Solution Proposal

The final implementation will:

- Replace static scripts with **interactive visualization**
- Use sliders to dynamically modify:
  - Kernel intensity
  - Filter strength

A key improvement is that the filters are not fixed anymore. Instead, the kernel is **dynamically adjusted**, allowing smooth transitions between:

- Original image
- Partially filtered
- Fully filtered

This enhances understanding of how parameters affect aesthetics.

Part 3: Individual Section – Serigne Mansour Diop

3.1. Summary of Studied Knowledge

This part focuses on **color quantization and edge detection**.

- **Posterize Filter**
  - Reduces the number of color levels
  - Transforms smooth gradients into flat regions
  - Produces a stylized, comic-like effect
- **Sobel Sketch Filter (Manual Implementation)**
  - Computes horizontal and vertical gradients using custom kernels
  - Gradient magnitude highlights edges
  - Final inversion produces a sketch-like rendering

3.2. Planned Technologies and Libraries

- Python
- NumPy (vectorized operations and gradient computation)
- OpenCV (image display and interaction)

3.3. Solution Proposal

The implementation includes:

- A **manual Sobel operator**:
  - Convolution with horizontal and vertical kernels
  - Gradient magnitude computation
- A **Posterize filter**:
  - Color quantization using intensity division

Interactive improvements:

- Slider for number of color levels
- Threshold control for edge filtering

This allows users to explore:

- Noise vs detail
- Weak vs strong edges
- Degree of abstraction

Part 4: Evaluation of Aesthetic Quality (NEW SECTION)

To address the subjective nature of "aesthetic effects", we introduce a structured evaluation approach:

4.1. Objective Metrics

- **Contrast** (standard deviation of intensity)
- **Sharpness** (gradient magnitude)
- **Entropy** (information content)

4.2. Visual Criteria

- Edge clarity
- Texture enhancement
- Color simplification
- Level of abstraction

4.3. Perceptual Interpretation

Each filter is associated with an artistic effect:

- Emboss → 3D relief perception
- Sharpen → clarity and detail enhancement
- Posterize → stylized/comic effect
- Sobel Sketch → hand-drawn appearance

Part 5: Literature and Sources Used

- Course materials on:
    - Spatial filtering
    - Edge detection
    - Image transformations
- Gonzalez, R. C., & Woods, R. E.
  Digital Image Processing, Pearson
- OpenCV documentation (used only for I/O and GUI, not core algorithms)